

Monte Carlo Simulation - Project

Emil Beck Aagaard, Kamille Møller Frost

October 9, 2025

1 Portfolio loss and importance sampling.

Let $V(S)$ be the value of a portfolio value consisting of financial instruments based on m market prices $S \in \mathbb{R}^m$. The goal of this project is to use Monte Carlo to estimate the portfolio loss L over short time periods ΔT . Here, it is common to use the delta-gamma representation, where the loss is written as

$$L := \delta^\top \Delta S + (\Delta S)^\top \Gamma \Delta S,$$

where $\delta \in \mathbb{R}^m$, $\Gamma \in \mathbb{R}^{m \times m}$ and $\Delta S \sim N(0, \Sigma_S)$ for some covariance matrix Σ_S . Sometimes, it is helpful to consider the linear approximation

$$Q := \delta^\top \Delta S.$$

First, assume that $m = 1$, $\delta = 2$, $\Gamma = \Sigma_S = 1$, and $x = 10$. Please always justify your answer.

1.1 Questions

Exercise a) Write a program to estimate the exceedance probabilities $\mathbb{P}(L > x)$ by crude Monte Carlo simulation.

Using the given parameters we can write:

$$\mathbb{P}(L > x) = \mathbb{P}(2Z + Z^2 > x) \quad Z \sim N(0, 1)$$

Completing the square we get $2Z + Z^2 = (Z + 1)^2 - 1 = Y^2 - 1$ where $Y \sim X_{nc}^2(\nu = 1, \lambda = 1)$ where $X_{nc}^2(\nu, \lambda)$ denotes a non-central chi-square distribution with ν degrees of freedom and non-centrality parameter λ . In particular the analytical probability is can be calculated by:

$$\mathbb{P}(L > x) = 1 - \mathbb{P}(Y^2 - 1 \leq x) = 1 - \mathbb{P}(Y^2 \leq x + 1)$$

In code:

```
[188]: x = 10
analytical_prop = 1-ncx2.cdf(x+1, df=1, nc=1)
print(f"Analytical probability P(L > {x}): {analytical_prop:.6f}")
```

Analytical probability P(L > 10): 0.010270

```
[189]: # assumptions
m = 1
delta = 2
gamma = 1
sigma = np.array([[1]]) # covariance matrix

# number of simulations
N = 10**4

def loss(m, delta, delta_S, gamma):
    """ Compute the loss L given delta, gamma, sigma and delta_S, note that all
    ↪ parameters should be of dimension m. """
    if m == 1:
        L = delta*delta_S + gamma*delta_S**2
    else:
        L = delta.T@delta_S + delta_S.T@gamma@delta_S
    return L

def crude_monte_carlo(N, m, delta, gamma, sigma, x, loss_function):
    """ Crude Monte Carlo simulation of the probability  $P(L > x)$ , note that all
    ↪ parameters should be of dimension m. """
    delta_S = np.random.multivariate_normal(np.zeros(m), sigma, N)
    L = loss_function(m, delta, delta_S, gamma)
    return np.mean(L > x), L

P_crude, L = crude_monte_carlo(N, m, delta, gamma, sigma, x, loss)
print(f"Crude Monte Carlo estimate of  $P(L > {x})$ : {P_crude:.6f}")
```

Crude Monte Carlo estimate of $P(L > 10)$: 0.010400

Exercise b) Use control variables method to reduce the variance of the estimator from part (a). Discuss several possibilities and argue for your final choice. Would you recommend to use these methods given potential longer compute time?

```
[190]: def estimator(m, delta, delta_S, *args):
    """Initially proposed linear estimator of L, note that all parameters should
    ↪ be of dimension m. """

    if m == 1:
        return delta*delta_S
    else:
        return delta.T@delta_S
```

```

P_crude_estimate, Q = crude_monte_carlo(N, m, delta, gamma, sigma, x, estimator)

def cv_estimator_1(Z, x, *args):
    """Simplest control variance estimator using Z as control variate."""
    H = Z > x/2
    return H, np.mean(H)

def cv_estimator_2(Z, x, *args):
    """Simplest control variance estimator using Z as control variate."""
    delta, m = args
    H = estimator(m, delta, Z, gamma)
    #print(f"Mean of control variate: {np.mean(H):.4f}")

    return H, np.mean(H)

def control_variance_monte_carlo(N, m, delta, x, control_variance_estimator):
    """ Control Variance monte carlo of  $P(L > x)$ , note that all parameters should
    be of dimension m. """

    # Create simulation
    Z = np.random.multivariate_normal(np.zeros(m), sigma, N)
    Y = loss(m, delta, Z, gamma) > x
    H, l_H = control_variance_estimator(Z, x, delta, m)

    model = LinearRegression()
    beta = model.fit(H, Y).coef_
    print(f"Optimal beta: {beta.item():.4f}")

    Y_hat = Y - beta*(H - l_H)
    if m == 1:
        return np.mean(Y_hat), Y, Y_hat
    else:
        pass

P_crude_cv_1, Y_1, Y_hat_1 = control_variance_monte_carlo(
    N, m, delta, x, cv_estimator_1)
print(f"Control Variance (estimator 1) Monte Carlo estimate of  $P(L > {x})$ :
    {P_crude_cv_1:.6f}")
print(f"Variance reduction: {np.var(Y_1)/np.var(Y_hat_1):.2f} times")

```

```

P_crude_cv_2, Y_2, Y_hat_2 = control_variance_monte_carlo(
    N, m, delta, x, cv_estimator_2)
print(f"Control Variance (estimator 2) Monte Carlo estimate of P(L > {x}):_
↪{P_crude_cv_2:.6f}")
print(f"Variance reduction: {np.var(Y_2)/np.var(Y_hat_2):.2f} times")

# create a plot of the convergence of the probability estimates
plt.figure(figsize=(10, 6))
plt.plot(np.cumsum(Y_hat_1)/np.arange(1, N+1), label='CV Estimator 1', alpha=0.
↪7)
plt.plot(np.cumsum(Y_hat_2)/np.arange(1, N+1), label='CV Estimator 2', alpha=0.
↪7)
plt.plot(np.cumsum(L > x)/np.arange(1, N+1), label='Crude MC', alpha=0.7)
plt.axhline(analytical_prop, color='red', linestyle='--', label='Analytical_
↪Probability')
plt.xlabel('Number of Simulations')
plt.ylabel(f'Estimate of P(L > {x})')
plt.title('Convergence of Monte Carlo Estimators')
plt.legend()
plt.grid()
plt.show()

```

Optimal beta: 0.0000

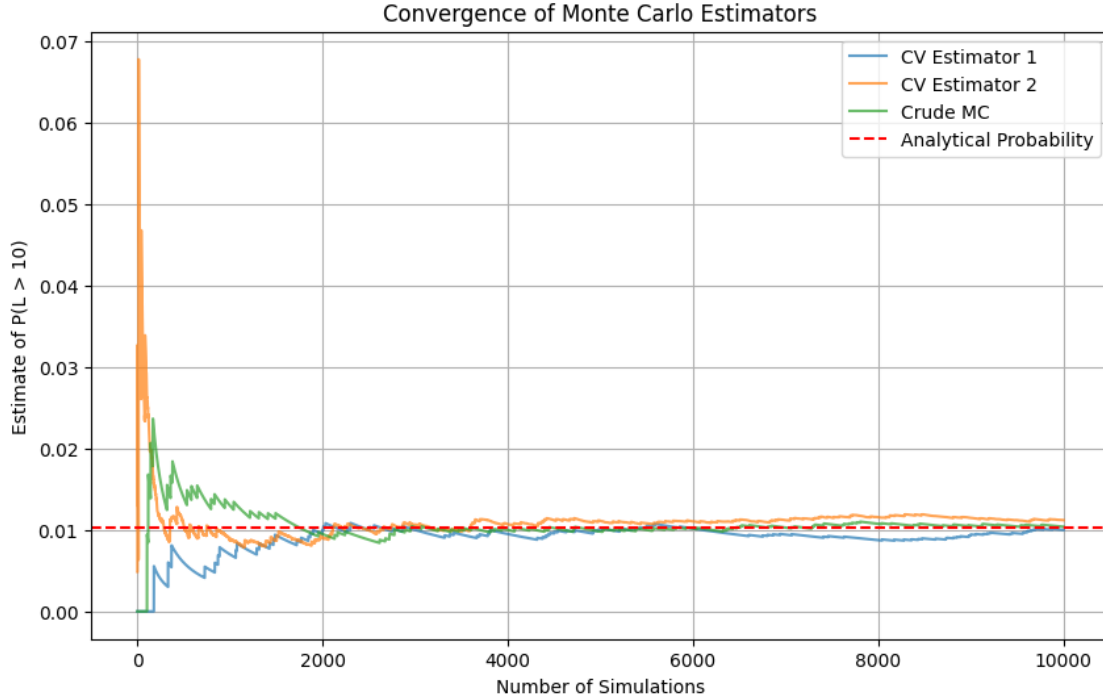
Control Variance (estimator 1) Monte Carlo estimate of P(L > 10): 0.010000

Variance reduction: 1.00 times

Optimal beta: 0.0150

Control Variance (estimator 2) Monte Carlo estimate of P(L > 10): 0.011200

Variance reduction: 1.09 times



Exercise c) Design an importance-sampling method to reduce the variance of the estimator from part (a). Again, discuss several options and argue for your final choice. Again, discuss the trade-off between additional compute time and variance reduction.

Exercise d) Create and discuss visualizations for comparing (a), (b) and (c).

Exercise e) Show that for the specific choices of m, δ, Γ and Σ_S , the probability $\mathbb{P}(L > x)$ has a closed-form expression only involving the cdf of the standard normal distribution. Is this still true when choosing another distribution? For the variance-reduction methods in (b), (c), how easy do they generalize when L is replaced by another relevant distribution?

Question 1) Now, assume that $m = 2, \delta = (2, 2)^\top, \Gamma = Id_2, \Sigma_S = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$, and $x = 15$. Repeat parts (a) and (b) for these new parameters.

Exercise f) Now, use the Cholesky decomposition to find a matrix $\tilde{C}$ such that $\Delta S = \tilde{C}Z$ where $Z \sim N(0, I_2)$. Next, find an orthogonal matrix U and a diagonal matrix $\Lambda = \text{diag}(\lambda_1, \lambda_2)$ such that $\tilde{C}^\top \tilde{C} = U\Lambda U^\top$. Show that the loss L can be expressed as

$$L = b_1 Z_1 + b_2 Z_2 + \lambda_1 Z_1^2 + \lambda_2 Z_2^2$$

where $b = C^\top \delta$ with $C = \tilde{C}U$.

Exercise g) Similarly as in part (c), use $\prod_{i \leq 2} \exp(\theta_i Z_i - \psi^*(\theta_i))$ as an importance-sampling density, where ψ^* is the cumulant generating function of a standard normal distribution. Experiment with different parameters θ_i .

Exercise h) For $(\lambda_1 \vee \lambda_2)\theta < 1/2$ show that

$$\psi_L(\theta) := \log \mathbb{E}[\exp(\theta L)] = \frac{1}{2} \sum_{j \leq 2} \left(\frac{\theta^2 b_j^2}{1 - 2\theta \lambda_j} - \log(1 - 2\theta \lambda_j) \right).$$

You may use without proof that

$$\mathbb{E} \left[\exp \left(\theta (Z_1 + c)^2 \right) \right] = (1 - 2\theta)^{-1/2} \exp(\theta c^2 / (1 - 2\theta))$$

holds for all $c \in \mathbb{R}$ and $\theta < 1/2$.

Exercise i) Use part (h) to develop an importance-sampling algorithm for the estimation of $\mathbb{P}(L > 15)$. Develop a heuristic for choosing θ .

Exercise j) Describe broadly how stratified sampling could be used for simulating L when the random variable Q is used for stratification. For this question, a description of your approach is sufficient; you do not need to implement a simulation program.